

IDAP

Insurance Damage Assessment Platform

Documentação de Design de Base de Dados

Modelo ER, Considerações de Design e Implementação em PostgreSQL

Alunos: Afonso Santos (n.º 50484)
Rodrigo Meneses (n.º 50542)

Orientadores: Artur Ferreira
Pedro Matutino

Curso: Licenciatura em Engenharia Informática e de Computadores

UC: Projeto e Seminário – Semestre de Verão 2025/26

Data: Março de 2026

Conteúdo

1	Modelo Entidade–Relacionamento	2
1.1	Visão Geral	2
1.2	Descrição das Entidades	2
2	Considerações de Design	3
2.1	Palavras-passe Nunca são Armazenadas em Plain Text	3
2.2	A Altura do Dano é um Valor Calculado	3
2.3	Polimorfismo de Evidências	3
3	Implementação em PostgreSQL	3
3.1	Exemplo de Tabelas	3
A	Diagrama EA	7

1 Modelo Entidade–Relacionamento

1.1 Visão Geral

O modelo ER está organizado em torno do conceito central de **AccidentCase** (Processo de Sinistro), que agrupa todas as entidades envolvidas numa única peritagem de seguros. A partir de um processo, o modelo ramifica-se em quatro áreas principais: utilizadores e autenticação, veículos e danos, evidências e medições, e análise e relatórios.

Nota: O diagrama detalhado do modelo Entidade-Relacionamento encontra-se disponível para consulta na última página deste documento (Anexo A).

1.2 Descrição das Entidades

Nota: É possível que vão existir mais tabelas derivadas de **Evidence** mas decidimos começar apenas pela análise de imagens.

Entidade	Descrição
User	Representa qualquer pessoa que se autentique na plataforma. Armazena credenciais (apenas hash da palavra-passe).
TOKENS	Armazena tokens de autenticação ativos. Suporta gestão de sessões e revogação de tokens.
AccidentCase	A entidade central. Representa um processo de sinistro individual.
WeatherConditions	Regista o contexto meteorológico no momento do acidente (temperatura, visibilidade, tipo de precipitação).
AccidentScene	Captura o contexto geométrico e ambiental do local: coordenadas GPS, inclinação do terreno, gradiente da estrada e tipo de via.
Vehicle	Um veículo envolvido num processo. Identificado pela marca, modelo, ano e matrícula.
Damage	Um dano específico observado num veículo. Regista a zona de contacto, tipo de deformação, direção do impacto e altura medida ao solo.
Evidence	Qualquer prova anexada ao processo (foto, vídeo, documento).
ImageEvidence	Extensão de Evidence para ficheiros de imagem. Armazena o caminho do ficheiro, dimensões e metadados (ex: dados EXIF).
Analysis	Uma sessão de análise formal realizada por um perito sobre um processo.
AnalysisImage	Tabela de junção que liga uma análise às imagens específicas utilizadas como prova.
Measurement	Armazena as coordenadas de pixéis de um objeto de referência e de uma área de dano para calcular a altura real.

Entidade	Descrição
DamageComparison	Regista a comparação formal entre dois danos de veículos diferentes.
AnalysisConclusion	A conclusão estruturada de uma análise: resultado de compatibilidade e fundamentação do perito.
Report	O documento final gerado para um processo.

Tabela 1: Descrição das entidades

2 Considerações de Design

2.1 Palavras-passe Nunca são Armazenadas em Plain Text

A tabela **User** armazena um campo `password_hash`. Antes de inserir ou atualizar um utilizador, a camada aplicacional deve cifrar a palavra-passe utilizando um algoritmo seguro como `bcrypt`.

2.2 A Altura do Dano é um Valor Calculado

A tabela **Measurement** armazena coordenadas de pixels. A altura real em centímetros *não* é armazenada na base de dados. Esta decisão foi tomada não só pelo facto de se um dia fosse necessário provar de onde veio essa medição não teria de ser feito todo o trabalho manual novamente, mas também a pensar na possibilidade da evolução do cálculo das medições. Caso a fórmula de cálculo seja refinada ou alargada com novos fatores, basta recalcular os valores a partir das coordenadas já armazenadas, sem que os dados históricos fiquem desatualizados ou inconsistentes.

2.3 Polimorfismo de Evidências

A entidade **Evidence** utiliza o padrão de *Class Table Inheritance* (Herança por Tabela de Classe). Existe uma tabela base **Evidence** com os atributos comuns a todas as evidências, e para cada subtipo é criada uma tabela separada — neste caso **ImageEvidence** — que referencia a tabela base e armazena apenas os atributos específicos desse subtipo (como dimensões e metadados EXIF). Esta abordagem distingue-se do *Single Table Inheritance*, onde todos os subtipos partilham uma única tabela com muitas colunas NULL; aqui cada tabela mantém apenas os dados que lhe são relevantes. As vantagens são: no caso de ser necessário adicionar outro tipo de evidência, o sistema continua a funcionar sem alterações à tabela base; as regras de integridade de cada subtipo podem ser aplicadas de forma rigorosa; e evita-se a proliferação de valores NULL.

3 Implementação em PostgreSQL

3.1 Exemplo de Tabelas

```

1
2 CREATE TABLE "User" (
3   user_id      SERIAL PRIMARY KEY,
4   name         VARCHAR(100) NOT NULL,

```

```
5     email          VARCHAR(255) NOT NULL UNIQUE ,
6     password_hash  VARCHAR(255) NOT NULL ,
7 );
8
9 CREATE TABLE TOKENS (
10     token_id       SERIAL PRIMARY KEY ,
11     user_id        INT NOT NULL REFERENCES "User"(user_id) ON DELETE
12         CASCADE ,
13     token          VARCHAR(512) NOT NULL UNIQUE ,
14     created_at     TIMESTAMP NOT NULL DEFAULT NOW() ,
15     expires_at     TIMESTAMP NOT NULL ,
16     revoked        BOOLEAN NOT NULL DEFAULT FALSE
17 );
```

Listing 1: Tabelas de Utilizador e Roles

```
1 CREATE TABLE AccidentCase (
2     case_id        SERIAL PRIMARY KEY ,
3     user_id        INT NOT NULL REFERENCES "User"(user_id) ,
4     created_at     TIMESTAMP NOT NULL DEFAULT NOW() ,
5     status         VARCHAR(50) NOT NULL DEFAULT 'open' ,
6     description    TEXT
7 );
8
9 CREATE TABLE WeatherConditions (
10     weather_id     SERIAL PRIMARY KEY ,
11     case_id        INT NOT NULL UNIQUE REFERENCES AccidentCase(
12         case_id) ON DELETE CASCADE ,
13     temperature    FLOAT ,
14     visibility      FLOAT ,
15     precipitation  VARCHAR(50)
16 );
17
18 CREATE TABLE AccidentScene (
19     scene_id       SERIAL PRIMARY KEY ,
20     case_id        INT NOT NULL UNIQUE REFERENCES AccidentCase(
21         case_id) ON DELETE CASCADE ,
22     latitude        FLOAT ,
23     longitude       FLOAT ,
24     road_type       VARCHAR(50) ,
25     road_gradient   FLOAT ,
26     terrain_slope   FLOAT
27 );
```

Listing 2: Tabela Central e Contexto do Acidente

```
1 CREATE TABLE Vehicle (
2     vehicle_id     SERIAL PRIMARY KEY ,
3     case_id        INT NOT NULL REFERENCES AccidentCase(case_id) ON
4         DELETE CASCADE ,
5     make           VARCHAR(100) NOT NULL ,
6     model          VARCHAR(100) NOT NULL ,
```

```

6   year          INT,
7   plate         VARCHAR(20)
8 );
9
10  CREATE TABLE Damage (
11     damage_id    SERIAL PRIMARY KEY,
12     vehicle_id   INT NOT NULL REFERENCES Vehicle(vehicle_id)
13                 ON DELETE CASCADE,
14     contact_zone  VARCHAR(100),
15     deformation_type VARCHAR(100),
16     impact_direction VARCHAR(50),
17     height_from_ground FLOAT
18 );

```

Listing 3: Veículos e Danos

```

1  -- Tabela base (atributos comuns a todas as evidências)
2  CREATE TABLE Evidence (
3     evidence_id  SERIAL PRIMARY KEY,
4     case_id      INT NOT NULL REFERENCES AccidentCase(case_id) ON
5                 DELETE CASCADE,
6     type         VARCHAR(50) NOT NULL, -- discriminador: 'image', '
7                 video', 'document'
8     uploaded_at  TIMESTAMP NOT NULL DEFAULT NOW(),
9     uploaded_by  INT REFERENCES "User"(user_id)
10 );
11
12 -- Subtipo: evidência de imagem
13 CREATE TABLE ImageEvidence (
14     evidence_id  INT PRIMARY KEY REFERENCES Evidence(evidence_id)
15                 ON DELETE CASCADE,
16     file_path    VARCHAR(512) NOT NULL,
17     width_px     INT,
18     height_px    INT,
19     exif_data    TEXT
20 );

```

Listing 4: Evidências – Class Table Inheritance

```

1  CREATE TABLE Analysis (
2     analysis_id  SERIAL PRIMARY KEY,
3     case_id      INT NOT NULL REFERENCES AccidentCase(case_id) ON
4                 DELETE CASCADE,
5     analyst_id   INT REFERENCES "User"(user_id),
6     created_at   TIMESTAMP NOT NULL DEFAULT NOW()
7 );
8
9 -- Tabela de junção N:M entre Analysis e ImageEvidence
10 CREATE TABLE AnalysisImage (
11     analysis_id  INT NOT NULL REFERENCES Analysis(analysis_id) ON
12                 DELETE CASCADE,
13     evidence_id  INT NOT NULL REFERENCES ImageEvidence(evidence_id)
14                 ON DELETE CASCADE,

```

```
12     PRIMARY KEY (analysis_id, evidence_id)
13 );
14
15 CREATE TABLE Measurement (
16     measurement_id      SERIAL PRIMARY KEY,
17     analysis_id          INT NOT NULL REFERENCES Analysis(
18         analysis_id) ON DELETE CASCADE,
19     ref_obj_x1           FLOAT, ref_obj_y1 FLOAT,
20     ref_obj_x2           FLOAT, ref_obj_y2 FLOAT,
21     damage_area_x1       FLOAT, damage_area_y1 FLOAT,
22     damage_area_x2       FLOAT, damage_area_y2 FLOAT
23 );
24
25 CREATE TABLE DamageComparison (
26     comparison_id SERIAL PRIMARY KEY,
27     analysis_id     INT NOT NULL REFERENCES Analysis(analysis_id) ON
28         DELETE CASCADE,
29     damage_id_a     INT NOT NULL REFERENCES Damage(damage_id),
30     damage_id_b     INT NOT NULL REFERENCES Damage(damage_id)
31 );
32
33 CREATE TABLE AnalysisConclusion (
34     conclusion_id      SERIAL PRIMARY KEY,
35     analysis_id        INT NOT NULL UNIQUE REFERENCES Analysis(
36         analysis_id) ON DELETE CASCADE,
37     compatibility       VARCHAR(50) NOT NULL,
38     justification       TEXT
39 );
40
41 CREATE TABLE Report (
42     report_id          SERIAL PRIMARY KEY,
43     case_id            INT NOT NULL UNIQUE REFERENCES AccidentCase(
44         case_id) ON DELETE CASCADE,
45     generated_at       TIMESTAMP NOT NULL DEFAULT NOW(),
46     file_path          VARCHAR(512)
47 );
```

Listing 5: Análise, Medições e Conclusão

A Diagrama EA

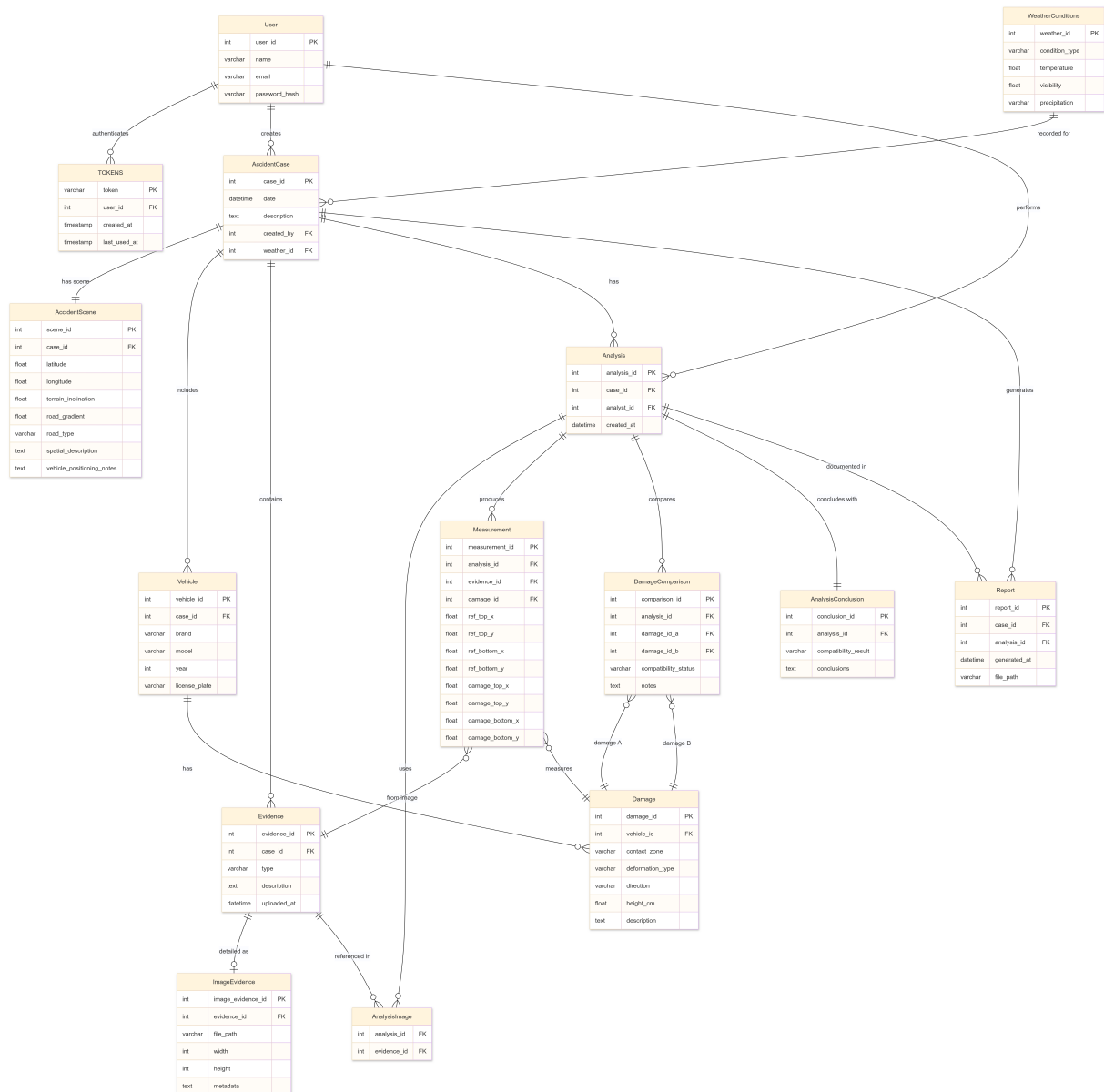


Figura 1: Modelo EA do sistema IDAP.

[Fim da Documentação]